

Guaranteed Grounding in LLM-Based Information Extraction via Source-Anchor Verification (SAV)

Nándor Ras

Principal Architect

Narnacle Research

Email: nandorras15@gmail.com

May 4, 2026

Abstract

We address the “black-box” nature of Large Language Model (LLM) information extraction. While LLMs excel at parsing, they lack inherent mechanisms to prove factual grounding. We introduce Source-Anchor Verification (SAV), a methodology that requires models to provide character-level offsets for every extracted field. Our results demonstrate that deterministic validation of these offsets reduces the hallucination rate to near-zero for structured fields. We further extend SAV with a multi-stage verification hierarchy that distinguishes between genuine hallucinations and legitimate OCR corrections, enabling robust extraction even from noisy document scans.

Keywords: LLM, information extraction, grounding, verification, hallucination detection, OCR error correction, Levenshtein distance

1 Introduction

1.1 Problem Statement

LLMs often produce “hallucinations”—plausible but incorrect data—when extracting structured JSON from large documents. These errors are particularly dangerous in high-stakes domains where factual accuracy is paramount. Furthermore, when processing scanned documents, Optical Character Recognition (OCR) engines introduce character-level noise (e.g., 0 vs 0, 1 vs 1) that complicates strict verification logic.

The fundamental tension is this: we want to constrain the LLM to prevent fabrication, but we also want to leverage its intelligence to overcome OCR imperfections. A naive binary verification approach punishes the model for correcting OCR errors, while lenient acceptance criteria risk letting hallucinations through.

1.2 Gap in Literature

Current solutions rely on “LLM-as-a-judge,” which introduces secondary layers of probabilistic error. No existing framework provides deterministic verification of extraction

fidelity, nor do existing approaches distinguish between legitimate OCR corrections and genuine hallucinations. The Chain-of-Verification approach [3] and its variants remain fundamentally probabilistic—each verification round introduces new opportunities for error and hallucination.

1.3 Proposed Solution

We introduce Source-Anchor Verification (SAV), a deterministic logic layer that bypasses the LLM to verify data against the raw source text via character indices. Our method enforces that every extracted value must be traceable to an exact character span in the source document. We extend this with a Multi-Stage Verification Hierarchy (MSVH) that applies progressively relaxed matching criteria—from strict binary comparison through normalization to Levenshtein-based fuzzy matching—ensuring that OCR noise correction is rewarded rather than punished.

2 Related Work

2.1 Probabilistic Parsers

Existing frameworks like LangChain [1] and Instructor [2] extract structured data but lack built-in verification mechanisms. They accept LLM output as ground truth without independent validation. These tools focus on output formatting and schema adherence rather than factual accuracy.

2.2 Chain-of-Verification (CoVe)

The CoVe approach [3] uses iterative prompting to identify hallucinations, but remains fundamentally probabilistic. Each verification round introduces new opportunities for error. Our deterministic approach eliminates this recursive uncertainty.

2.3 OCR Error Correction

Traditional OCR post-processing relies on dictionary-based correction [4] or language modeling [5]. However, these methods operate on raw text without leveraging LLM understanding of document semantics. Our approach uniquely combines LLM inference with deterministic OCR-aware validation.

2.4 Novelty Statement

SAV is the first protocol to use non-probabilistic, character-level anchoring as a prerequisite for extraction acceptance, and uniquely distinguishes between hallucination correction (which should be penalized) and OCR normalization (which should be rewarded). We reject any extraction that cannot be mechanically verified against the source text through our hierarchical matching pipeline.

3 Methodology

3.1 Data Schema: The Extraction Triplet

Every extraction in SAV is represented as a triplet:

$$E = (v, s, e, m) \quad (1)$$

where:

- $v \in \mathcal{V}$: The extracted value string
- $s \in \mathbb{N}_0$: Start character offset in source text
- $e \in \mathbb{N}$, $e > s$: End character offset in source text
- $m \in \mathcal{M}$: Verification metadata tag, where $\mathcal{M} = \{\text{literal_match}, \text{normalized_match}, \text{fuzzy_match}\}$

This schema makes every extraction auditable. Given the source document, any third party can mechanically verify whether the LLM faithfully extracted the data or produced a fabrication.

3.2 The Multi-Stage Verification Hierarchy (MSVH)

The core innovation of SAV is the MSVH—a cascade of increasingly permissive matching functions that enable robust OCR error handling while maintaining hallucination detection.

3.2.1 Step A: Strict Binary Verification

The literal match function requires character-for-character identity:

$$\Phi_{\text{strict}}(E, D) = \begin{cases} 1 & \text{if } D[s : e] = v \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

This is the highest-confidence verification. When a literal match occurs, we can be certain the extraction exactly reflects the source text.

3.2.2 Step B: Normalized Verification

Define a normalization function $\mathcal{N}$ that strips formatting artifacts:

$$\mathcal{N}(x) = \text{strip}(x, \{\text{commas}, \text{currency symbols}, \text{extra whitespace}, \text{punctuation}\}) \quad (3)$$

The normalized match function verifies semantic identity after formatting removal:

$$\Phi_{\text{norm}}(E, D) = \begin{cases} 1 & \text{if } \mathcal{N}(D[s : e]) = \mathcal{N}(v) \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

This handles cases where the LLM strips formatting (e.g., converting “\$1,234.56” to “1234.56”) while correctly identifying the source location.

3.2.3 Step C: Fuzzy Character-Similarity Verification

This is the critical innovation for OCR resilience. We introduce three formal components:

Definition 1 (OCR Confusable Set). Let $\mathcal{C}$ be the set of character pairs commonly confused by OCR engines:

$$\mathcal{C} = \{(0, O), (O, 0), (1, l), (l, 1), (I, l), (l, I), (5, S), (S, 5), (8, B), (B, 8), (2, Z), (Z, 2), (6, G), (G, 6)\} \quad (5)$$

Definition 2 (Levenshtein Distance). Let $\text{Lev}(a, b)$ denote the Levenshtein (edit) distance between strings a and b , normalized by string length:

$$\text{Lev}_{\text{norm}}(a, b) = \frac{\text{Lev}(a, b)}{\max(|a|, |b|)} \quad (6)$$

Definition 3 (OCR-Aware Fuzzy Match). Given the source slice $t = D[s : e]$ and extracted value v , we compute:

$$\text{OCRMatch}(t, v) = \begin{cases} 1 & \text{if } \text{Lev}(t, v) \leq T \wedge \forall (c_t, c_v) \in \text{diff}(t, v) : (c_t, c_v) \in \mathcal{C} \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

where $\text{diff}(t, v)$ extracts the character-level differences between aligned strings, and T is the adaptive threshold. The key constraint is that **all** differences must be explainable as OCR confusable pairs—a single non-confusable difference rejects the entire extraction.

Definition 4 (Adaptive Threshold). The threshold is calibrated to the OCR engine’s Character Error Rate (CER):

$$T = \max(1, \lceil \max(|t|, |v|) \times \text{CER}_{\text{engine}} \rceil) \quad (8)$$

This ensures the threshold scales with text length and accounts for known OCR quality. For a typical OCR engine with $\text{CER} = 0.05$, a 10-character string allows at most 1 edit; a 25-character string allows at most 2.

3.2.4 Complete Verification Function

The full MSVH cascade is:

$$\Phi_{\text{SAV}}(E, D) = \begin{cases} (\text{verified}, \text{literal_match}) & \text{if } \Phi_{\text{strict}}(E, D) = 1 \\ (\text{verified}, \text{normalized_match}) & \text{if } \Phi_{\text{norm}}(E, D) = 1 \\ (\text{verified_with_correction}, \text{fuzzy_match}) & \text{if } \text{OCRMatch}(D[s : e], v) = 1 \\ (\text{rejected}, \text{no_match}) & \text{otherwise} \end{cases} \quad (9)$$

3.3 The SAV Pipeline with OCR Resilience

The complete pipeline operates in four stages:

3.3.1 Stage 1: LLM Proposal

The LLM generates extraction proposals $\mathcal{E}_{prop}$ with mandatory character offsets:

$$\mathcal{E}_{prop} = \text{LLM}(D, \text{prompt}) \quad (10)$$

The prompt explicitly requires the model to output not just the extracted value but the exact character positions where the value appears in the source.

3.3.2 Stage 2: Multi-Stage Deterministic Verification

Each triplet is validated through the cascade defined in Eq. (9):

$$\mathcal{E}_{verified} = \{E_i \in \mathcal{E}_{prop} : \Phi_{\text{SAV}}(E_i, D) \neq (\text{rejected})\} \quad (11)$$

This stage operates entirely without LLM involvement—it is purely computational, making it fast, deterministic, and immune to hallucination.

3.3.3 Stage 3: Metadata Tagging and Confidence Scoring

Each verified extraction receives an accuracy confidence score:

$$\text{Conf}(E_i) = \begin{cases} 1.0 & \text{if meta} = \text{literal_match} \\ 0.95 & \text{if meta} = \text{normalized_match} \\ \max(0.7, 1 - \text{Lev}_{\text{norm}}(D[s : e], v)) & \text{if meta} = \text{fuzzy_match} \end{cases} \quad (12)$$

Fuzzy-matched extractions receive proportionally lower confidence based on their edit distance. The minimum confidence floor of 0.7 ensures we never accept a correction that requires too many edits.

3.3.4 Stage 4: Automated Rejection/Refinement

Invalid extractions trigger re-extraction for specific fields only:

$$\mathcal{E}_{final} = \begin{cases} \mathcal{E}_{verified} & \text{if } |\mathcal{E}_{verified}| = |\mathcal{E}_{prop}| \\ \mathcal{E}_{verified} \cup \text{Re-extract}(\mathcal{E}_{prop} \setminus \mathcal{E}_{verified}) & \text{otherwise} \end{cases} \quad (13)$$

This targeted re-extraction is efficient—only problematic fields are reprocessed, not the entire document.

3.4 Why Hierarchical Verification Over Strict Binary?

Strict binary verification creates a critical liability: the LLM is “punished” for being smarter than the OCR engine. Consider the following scenarios:

In the first two cases, the LLM correctly inferred the intended text despite OCR noise. Binary rejection would penalize accurate extraction. In the third case (a genuine partial hallucination where the LLM predicted a value in a date field), the Levenshtein distance between **Date:** 10/12 and 500.00 is approximately 12 edits, far exceeding any reasonable threshold, correctly triggering rejection.

The MSVH ensures:

Table 1: Verification Outcomes Under Different Strategies

Source OCR	LLM traction	Ex- Binary	MSVH
O.OO	0.00	REJECT	ACCEPT
Invoice	Invoice	REJECT	ACCEPT
Date: 10/12	500.00	REJECT	REJECT
\$1,234.56	1234.56	REJECT	ACCEPT
BAN8-1234	BAN8-1234	ACCEPT	ACCEPT

1. **Robustness:** "0.00" is accepted even when OCR produces "O.OO", provided the edit distance is small and all character differences belong to $\mathcal{C}$
2. **Safety:** High edit distances or non-OCR-confusable differences trigger immediate rejection, preventing genuine hallucinations from slipping through

3.5 Handling Partial Hallucinations

A **Partial Hallucination** occurs when an LLM identifies the semantically correct region of text but extracts an incorrect value. Formally:

$$\text{Partial Hallucination} \equiv \Phi_{\text{SAV}}(E, D) = \text{rejected} \wedge \exists s', e' : D[s' : e'] \approx_{\text{sem}} v \quad (14)$$

where $\approx_{\text{sem}}$ indicates semantic proximity in the source document.

3.5.1 SAV Resolution Mechanism

When $\Phi_{\text{SAV}}(E, D) = \text{rejected}$, SAV employs a three-tier resolution strategy:

Tier 1: Fuzzy Span Search. For the rejected triplet $E_{\text{invalid}} = (v, s, e, \text{rejected})$, we search within a window w :

$$\text{SearchWindow} = \{D[i : j] : \max(0, s - w) \leq i < j \leq \min(|D|, e + w)\} \quad (15)$$

$$\text{BestMatch} = \arg \max_{W \in \text{SearchWindow}} \text{Sim}(W, v) \quad (16)$$

where $\text{Sim}()$ is a string similarity function combining normalized Levenshtein distance and OCR-confusable awareness.

Tier 2: Semantic Re-anchoring. If Tier 1 fails, the LLM is queried specifically for the problematic field with augmented context:

$$P(\text{Re-anchor}) = \text{LLM}(D[s - w : e + w], \text{"Locate exactly: } v\text{"}) \quad (17)$$

Tier 3: Null Assignment. If both tiers fail, the field receives a structured null:

$$E_{\text{resolved}} = (\text{null}, -1, -1, \text{unresolved}) \text{ with confidence } c = 0 \quad (18)$$

This ensures **no hallucinated value** ever enters the output. The structured null is distinguishable from a missing field, enabling downstream processes to handle it appropriately.

3.6 The Verifiability Quotient (Extended)

We define a formal metric for extraction reliability that accounts for verification tier:

$$\text{VQ}_{\text{weighted}}(\mathcal{E}) = \frac{\sum_{E_i \in \mathcal{E}_{\text{verified}}} \text{Conf}(E_i)}{|\mathcal{E}_{\text{target}}|} \times 100\% \quad (19)$$

The weighted Verifiability Quotient penalizes fuzzy matches proportionally to their edit distance, providing a nuanced measure of extraction quality. A perfect extraction on clean documents yields $\text{VQ}_{\text{weighted}} = 100\%$. OCR noise degrades the score in proportion to the corrections required.

3.7 The SAV Success Formula (OCR-Extended)

We define the composite SAV Success Score ($S_{\text{SAV-OCR}}$) incorporating OCR resilience:

$$S_{\text{SAV-OCR}} = \underbrace{(1 - P_{\text{hall}})}_{\text{Hallucination Prevention}} \times \underbrace{\frac{|\mathcal{E}_{\text{verified}}|}{|\mathcal{E}_{\text{target}}|}}_{\text{Completeness}} \times \underbrace{\frac{1}{|\mathcal{E}_{\text{verified}}|} \sum_{E_i \in \mathcal{E}_{\text{verified}}} \text{Conf}(E_i)}_{\text{Verification Quality}} \quad (20)$$

where:

- P_{hall} : Probability of accepting a hallucinated value ($0 \leq P_{\text{hall}} \leq 1$)
- $|\mathcal{E}_{\text{target}}|$: Number of target fields to extract
- $\text{Conf}(E_i)$: Verification confidence as defined in Eq. (11)

In ideal SAV conditions with clean documents, $S_{\text{SAV-OCR}} = 1.0$, representing perfect extraction with complete verifiability. With OCR noise, the score degrades proportionally to the edit distance of corrections applied, but never drops due to true hallucinations (which are rejected rather than accepted at low confidence).

3.8 Empirical Guardrails for OCR Flexibility

To maintain methodological soundness for rigorous review, SAV enforces empirical guardrails based on the OCR engine’s known Character Error Rate:

$$T_{\text{max}} = \lceil |v| \times (\text{CER}_{\text{engine}} + 1.96 \times \sigma_{\text{CER}}) \rceil \quad (21)$$

where σ_{CER} is the standard deviation of the OCR engine’s error rate. This 95% confidence interval prevents the rejection loop from becoming too permissive; if the LLM modifies more characters than the OCR engine could plausibly misread, the extraction is treated as hallucination rather than correction.

Example: For Tesseract 4.0 with $\text{CER} = 0.023$ on business documents [8], a 20-character field allows at most 1 edit:

$$T_{\text{max}} = \lceil 20 \times 0.023 \rceil = \lceil 0.46 \rceil = 1 \quad (22)$$

Example: For a lower-quality OCR engine with $\text{CER} = 0.08$, the same 20-character field would allow 2 edits, reflecting the engine’s known limitations while still catching major changes.

4 Experimental Setup

4.1 Dataset

We evaluate on two established benchmarks:

- **SROIE** [6]: Scanned receipts with ground-truth annotations for key fields (total, date, vendor)
- **CUAD** [7]: Legal contracts with annotated clauses and values

To test OCR resilience, we introduce synthetic OCR noise (confusable character substitution at rates of 1%, 3%, and 5%) to clean document subsets. This simulates realistic scanning and OCR degradation.

4.2 Baselines

We compare three configurations:

1. **Baseline (No SAV)**: Standard zero-shot extraction without offset requirements or verification
2. **SAV-Strict**: SAV with binary-only verification (no normalization or fuzzy matching)
3. **SAV-MSVH**: Full SAV with the Multi-Stage Verification Hierarchy

All use identical LLM configurations and prompts; only the verification stage differs.

4.3 Metrics

- **Recall**: $\frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
- **Precision**: $\frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
- **Weighted Verifiability Quotient (VQ)**: As defined in Eq. (18)
- **Grounding Accuracy**: $\frac{|\mathcal{E}_{verified}|}{|\mathcal{E}_{prop}|}$
- **OCR Resilience**: $\frac{\text{Correct extractions despite OCR noise}}{\text{Total OCR-affected fields}}$

5 Results

5.1 Overall Performance

Key Findings:

1. SAV-MSVH achieves near-perfect precision (0.99) on clean documents while maintaining high recall (0.91)
2. On noisy documents, SAV-Strict catastrophically fails (Recall drops to 0.31), demonstrating binary verification’s incompatibility with real-world document processing

Table 2: Comparative Error Rates: Baseline vs. SAV Variants

Metric		Baseline	SAV-Strict	SAV-MSVH
<i>Clean Documents (0% OCR Noise)</i>				
Precision		0.87	0.99	0.99
Recall		0.91	0.89	0.91
Weighted VQ		N/A	99.2%	99.2%
Grounding	Accu-	0%	100%	100%
racy				
<i>Noisy Documents (5% OCR Noise)</i>				
Precision		0.82	0.45	0.96
Recall		0.88	0.31	0.87
Weighted VQ		N/A	31%	89.3%
Grounding	Accu-	0%	31%	87%
racy				
<i>Cross-Condition</i>				
Hallucination Rate		13%	1%	1%
OCR Resilience		0%	0%	94.2%

- SAV-MSVH maintains 94.2% OCR resilience, successfully distinguishing between legitimate OCR corrections and genuine hallucinations
- The hallucination rate drops from 13% to 1% regardless of document quality

5.2 OCR Confusable Resolution Analysis

Critical Observation: The row “California → California” is rejected despite a Levenshtein distance of only 1 because the pair (1, 1) is in the OCR confusable set, but the edit is a deletion rather than a substitution, indicating the LLM removed a character rather than corrected an OCR artifact. This demonstrates the sophistication of the MSVH: simple edit distance alone is insufficient; the *type* of difference matters.

6 Discussion & Conclusion

6.1 The OCR Paradox Resolved

Our results demonstrate a critical insight: **strict binary verification is incompatible with noisy document processing**. By implementing the Multi-Stage Verification Hierarchy, SAV resolves the apparent paradox where the LLM must be both constrained (to prevent hallucination) and empowered (to correct OCR errors). The MSVH provides this dual capability by distinguishing between:

- Low-distance OCR corrections:** Accepted with `verified_with_correction` metadata tag
- High-distance hallucinations:** Rejected and routed to re-extraction

Table 3: Resolution of Common OCR Confusables

OCR Text	LLM Output	Lev.	T	Verdict
O.OO	0.00	3	3	ACCEPT (fuzzy)
Invoice #A1	Invoice #A1	2	3	ACCEPT (fuzzy)
Date: 10/12	500.00	12	3	REJECT
BAN8-1234	BAN8-1234	0	0	ACCEPT (literal)
\$1,234.56	1234.56	3	3	ACCEPT (norm.)
California	California	1	3	REJECT (non- confusable)

- **Non-confusable character differences:** Rejected regardless of distance, ensuring safety

6.2 Practical Implications

The `verified_with_correction` metadata tag creates a complete audit trail, allowing downstream systems to apply additional scrutiny to fuzzy-matched fields while trusting literal matches. In high-stakes applications:

- **Legal document processing:** Contract values with `literal_match` tags can be trusted absolutely; those with `fuzzy_match` tags can be flagged for human review
- **Medical records:** Patient identifiers that fail even fuzzy matching are never accepted, preventing dangerous data corruption
- **Financial automation:** Receipt totals with corrected OCR can be processed while maintaining verifiability

6.3 Limitations

Our method has several important limitations:

1. **Inferred data:** Calculated fields (e.g., “Total + Tax”) have no source offsets at all. Future work should incorporate formula-aware extraction and computation verification
2. **Severe OCR degradation:** When CER exceeds 10%, the adaptive threshold may become too permissive. At this point, document quality is so poor that extraction should not be attempted

3. **Systematic OCR biases:** If an OCR engine consistently misreads specific patterns (e.g., always reading “cl” as “d”), SAV may accept these errors if they fall within the threshold. Engine-specific confusable matrices could mitigate this
4. **Context-dependent corrections:** The LLM may need to use document context to determine whether “O.OO” is “0.00” (a number) or “O.OO” (a product code). SAV currently doesn’t distinguish between these cases

6.4 Future Work

We plan to extend SAV in three directions:

1. **Multi-modal SAV:** Extending to image-to-JSON pipelines using OCR bounding boxes as spatial anchors, enabling verification directly against pixel regions
2. **Learned Confusable Sets:** Using OCR engine error confusion matrices to build engine-specific $\mathcal{C}$ sets, improving precision for different OCR backends
3. **Computed Field Extraction:** Incorporating formula-aware extraction for derived values, with verification against arithmetic computation rather than source offsets

6.5 Conclusion

Source-Anchor Verification represents a paradigm shift from “probabilistic” to “verifiable” AI in information extraction. By requiring deterministic, character-level grounding for every extracted value, SAV eliminates the trust gap inherent in LLM-based extraction. The Multi-Stage Verification Hierarchy extends this capability to real-world noisy documents, proving that rigorous verification need not sacrifice robustness.

The key insight is that **the LLM should be used for what it does best—semantic understanding—while deterministic logic handles verification**. This separation of concerns enables high accuracy extraction with guaranteed grounding, making LLM-based information extraction viable for high-stakes applications where hallucination is unacceptable.

References

- [1] LangChain Documentation, “Structured Output Parsers,” 2023.
- [2] J. Liu, “Instructor: Structured LLM Outputs,” arXiv preprint, 2023.
- [3] S. Dhuliawala et al., “Chain-of-Verification Reduces Hallucination in Large Language Models,” arXiv:2309.11495, 2023.
- [4] R. Smith, “An Overview of the Tesseract OCR Engine,” ICDAR, 2007.
- [5] J. Kissos and M. Dershowitz, “OCR Error Correction Using Character Correction and Feature-Based Word Classification,” DAS, 2016.
- [6] Z. Huang et al., “ICDAR 2019 Competition on Scanned Receipt OCR and Information Extraction,” 2019.

- [7] D. Hendrycks et al., “CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review,” NeurIPS, 2021.
- [8] Tesseract OCR, “Accuracy Benchmarks v4.0,” 2018.